

《计算机网络》实验报告

实验 3：配置 Web 服务器 捕获 HTTP 报文并分析

姓名：蒋衲言 学号：2313546

1 实验内容及要求

1. 搭建 Web 服务器（自由选择系统），并制作简单的 Web 页面，包含简单文本信息（包含专业、学号、姓名）和 6 幅图像。
2. 通过浏览器获取自己编写的 Web 页面，使用 Wireshark 捕获浏览器与 Web 服务器的交互过程。
3. 分析两个报文的封装层次，包括数据链路层、互连层、传输层、应用层。
4. 对整个 HTTP 交互过程进行详细说明，使用 Wireshark 过滤器使其只显示 HTTP 协议。
5. 提交 HTML 文档、Wireshark 捕获文件和实验报告。
6. 页面不要太复杂，包含所要求的基本信息即可。使用 HTTP，不使用 HTTPS。

2 环境配置与服务器搭建

2.1 环境配置

2.1.1 服务器

本次实验服务端的操作系统我选择 Linux **Ubuntu**（版本 24.04）。Ubuntu 系统运行在 VMware 虚拟机里。在 VMware 里将虚拟机的网络设置为 NAT 模式，如图1所示。

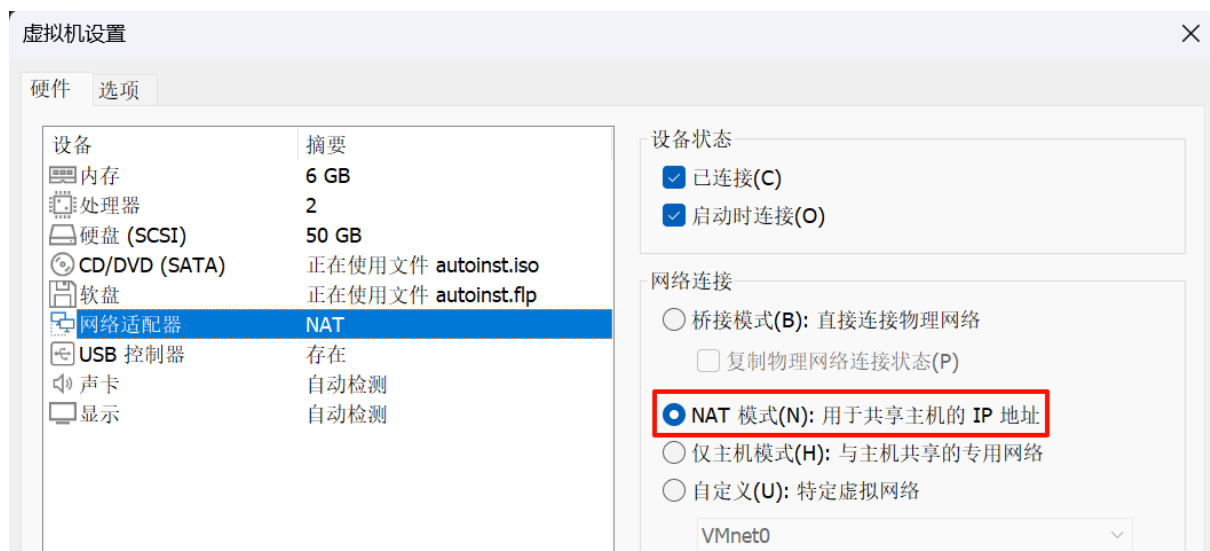


图 1: Ubuntu 虚拟机的网络设置

打开终端，执行命令 `ip addr show`，得到如图2所示的结果。

```
jry@jry-VMware-Virtual-Platform: ~/桌面
jry@jry-VMware-Virtual-Platform:~/桌面$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:1d:20:d2 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.188.132/24 brd 192.168.188.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 6e:00:6a:d9:26:17 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```

图 2: Ubuntu 虚拟机的网络信息

其中 lo 是回环接口，docker0 是 Docker 虚拟网桥，二者在本次实验可以忽略；而 ens33 才是虚拟机的“真正网卡”。将有用的信息在表1中罗列出来。

表 1: Ubuntu 虚拟机的网络信息

MAC 地址	IPv4 地址	子网掩码	网段
00:0c:29:1d:20:d2	192.168.188.132	255.255.255.0	192.168.188.0/24

2.1.2 客户端

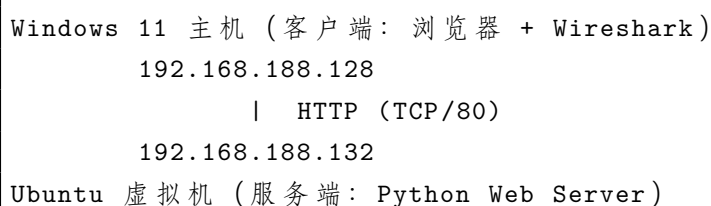
客户端是 Windows 11 系统，执行 `ipconfig`，得到客户端的 IP 地址为 192.168.188.128，如图3所示。

```
以太网适配器 VMware Network Adapter VMnet8:

    连接特定的 DNS 后缀 . . . . . :
    本地链接 IPv6 地址 . . . . . : fe80::67b4:a603:6795:6a79%10
    IPv4 地址 . . . . . : 192.168.188.128
    子网掩码 . . . . . : 255.255.255.0
    默认网关 . . . . . : 192.168.188.2
```

图 3: Windows 11 主机的 IP 地址

本次实验的拓扑结构如下：



2.2 编写 Web 页面与服务器搭建

本次实验服务器选择 Python 自带的 HTTP 服务器——Python `http.server`。

依次执行以下命令准备实验目录：

```
bash
```

```
mkdir ~/netlab_web  
cd ~/netlab_web  
mkdir images
```

编写以下 HTML 页面，保存为 `index.html`：

```
html
```

```
<!DOCTYPE html>  
<html lang="zh-CN">  
<head>  
  <meta charset="UTF-8">  
  <title>《计算机网络》实验3 Web 页面</title>  
</head>  
<body>  
  <h1>计算机网络实验3：配置 Web 服务器，捕获 HTTP 报文并分析</h1>  
  <p>专业：信息安全</p>  
  <p>学号：2313546</p>  
  <p>姓名：蒋衲言</p>  
  
  <h2>实验图片</h2>  
    
    
  <br>  
    
    
    
</body>  
</html>
```

随便找了 6 张图片，放到 `images` 文件夹下，形成如下的结构：

```
netlab_web/  
├── index.html  
└── images/  
    ├── 1.jpg  
    ├── 2.jpg  
    ├── 3.jpg  
    └── 4.jpg
```

5.jpg
6.jpg

在 netlab_web 目录下打开终端，执行 `python3 -m http.server 8000` 启动服务器。当前目录就是 Web 根目录，故 index.html 就是默认访问的 HTML 页面。除此之外，8000 端口是非特权端口，不需要 sudo 权限，所以比 80 端口更加便捷。

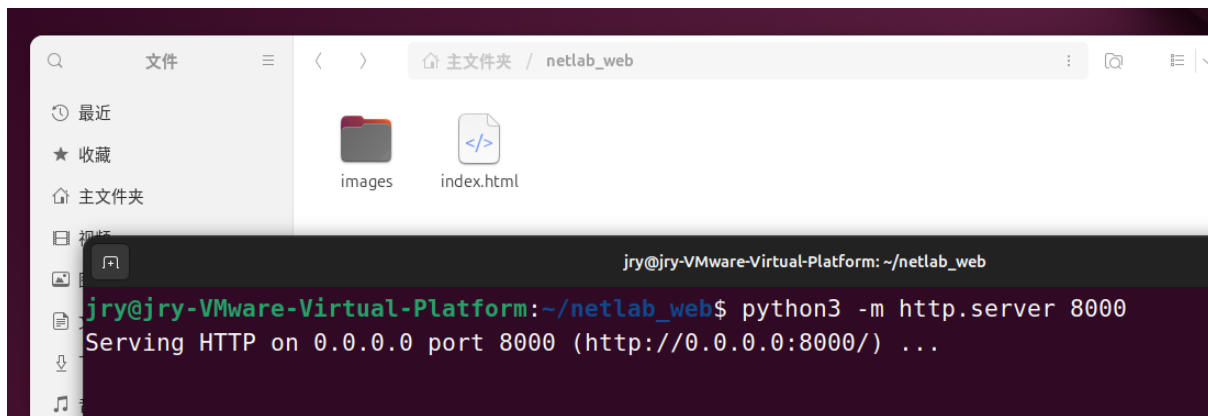


图 4: 启动服务器

2.3 使用 Wireshark 抓包

客户端打开 Wireshark，选择 VMware Network Adapter VMnet8 网卡（如图5所示），开始抓包。

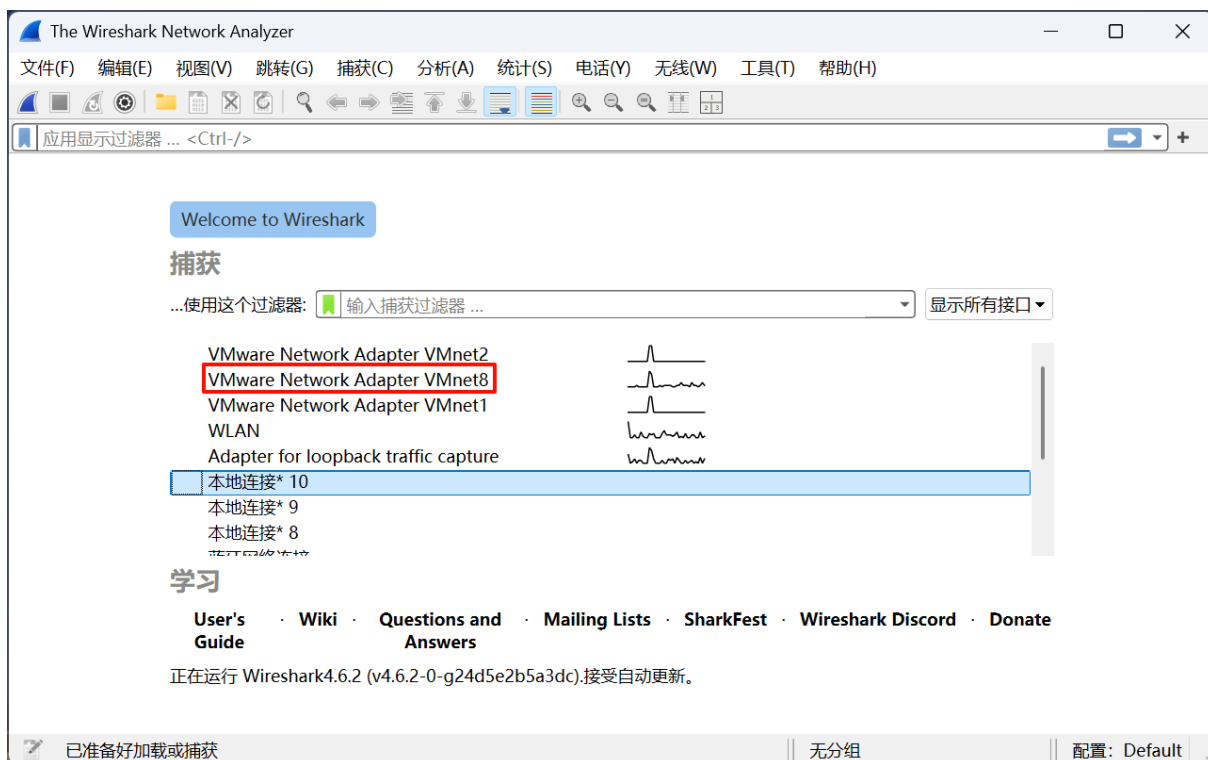


图 5: 选择 VMware Network Adapter VMnet8 网卡

客户端打开浏览器，访问`http://192.168.188.132:8000/`，可以看到出现了刚刚编写的HTML 页面，如图6所示。



图 6: HTML 页面

可以看到 Wireshark 里抓到了很多包，使用过滤条件`ip.addr == 192.168.188.132`进行过滤。过滤后的包如图7所示。

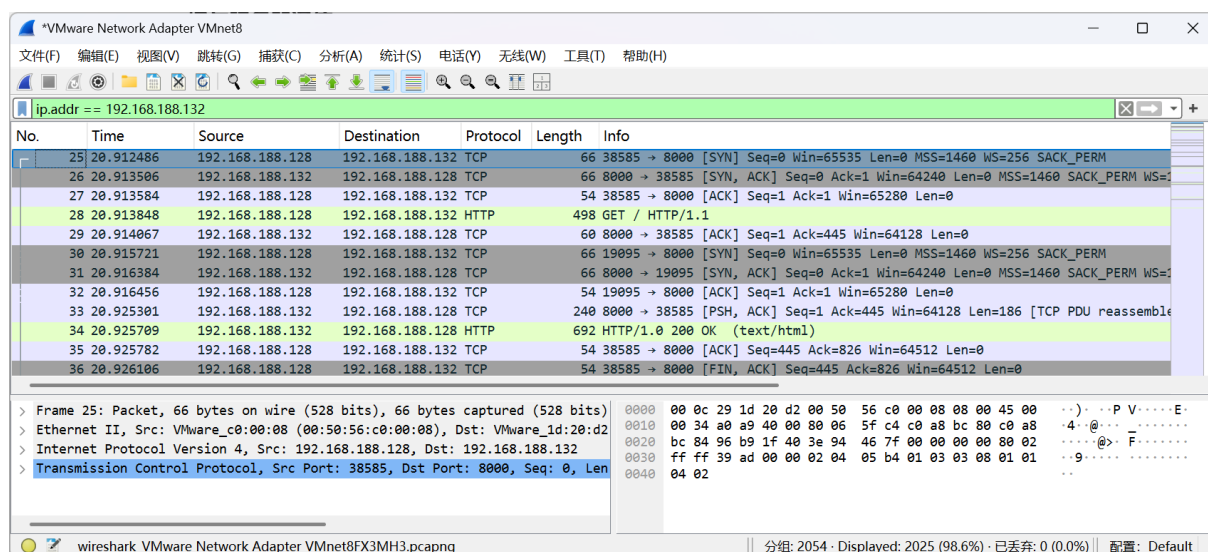


图 7: 过滤后的数据包

3 数据包的报文封装层次分析

选择一个 HTTP 请求包和 HTTP 响应包进行分析，如图8和9所示。HTTP 请求包是由客户端发往服务器的，响应包是服务器发往客户端的，二者封装层次类似。

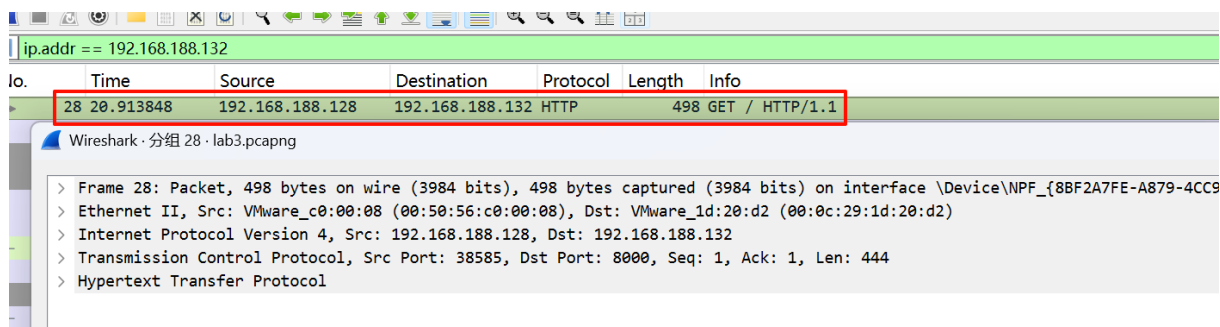


图 8: HTTP 请求包的封装层次

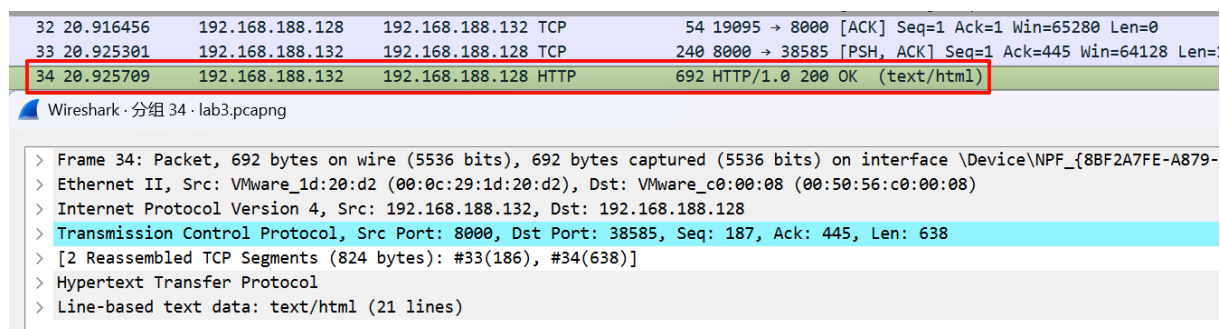
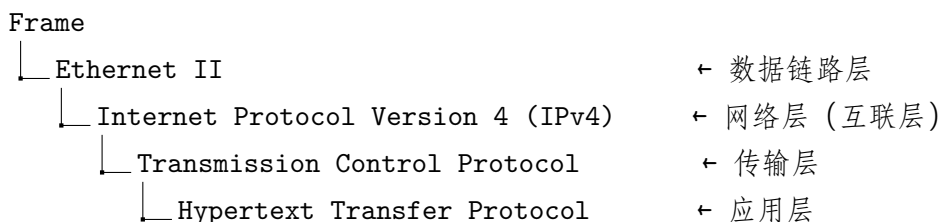


图 9: HTTP 响应包的封装层次

可以看到 HTTP 报文的协议层次如下：



3.1 数据链路层

需要注意的是，本次实验要求分析报文的数据链路层，故不能让客户端和服务端位于同一主机然后用回环地址 127.0.0.1 访问，因为 Loopback 是“逻辑接口”，不是物理网卡，通信不经过任何物理或虚拟网卡，操作系统直接在 IP 层内部转发数据包，所以根本没有 Ethernet 帧产生。

首先查看 HTTP 请求包的数据链路层封装，数据包的前 14 个字节属于 Ethernet II 帧头，如图10所示。

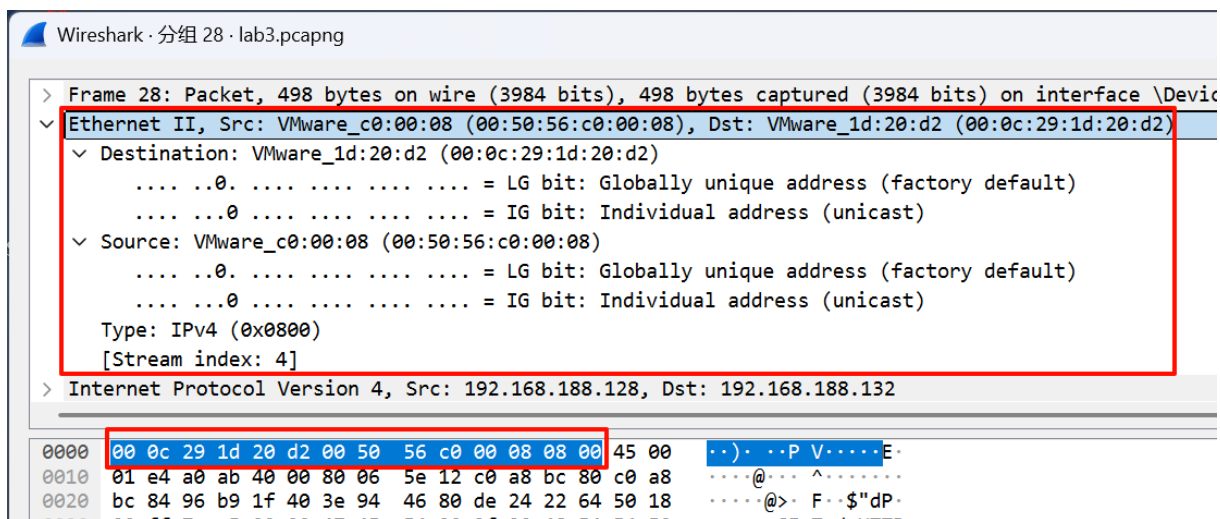


图 10: HTTP 请求包的 Ethernet II 帧头

Src: VMware_c0:00:08 (00:50:56:c0:00:08)是源 MAC 地址, 这里是 Windows 主机的 VMware NAT 网卡。

Dst: VMware_1d:20:d2 (00:0c:29:1d:20:d2)是目的 MAC 地址, 这里是 Ubuntu 虚拟机的虚拟网卡。

LG bit: Globally unique address (factory default)表示 LG (Local / Global) 位是 0, 表明这是全球唯一 MAC 地址。

IG bit: Individual address (unicast)表示 IG (Individual / Group) 位是 0, 表明这是单播地址 (Unicast)。

Type: IPv4 (0x0800)表示以太网帧的数据部分封装的是 IPv4 数据包。

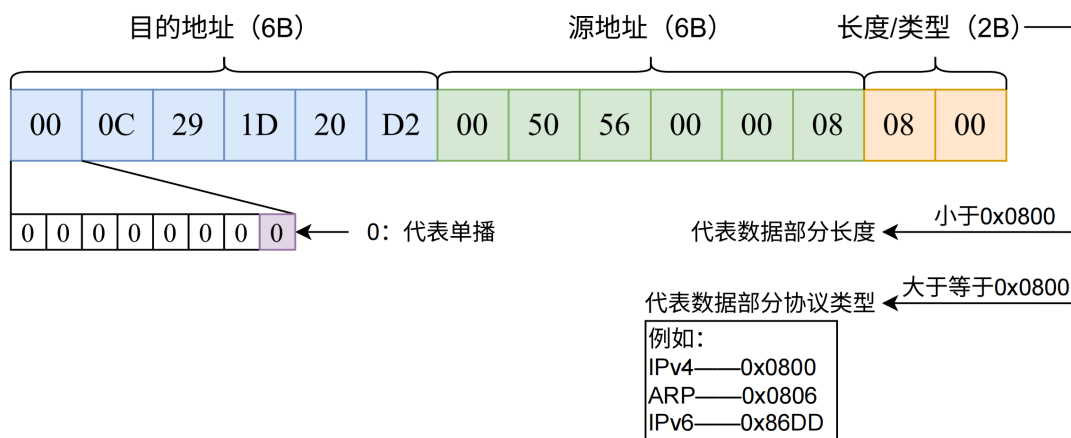


图 11: Ethernet II 帧头分析

接着查看 HTTP 响应包的数据链路层封装, 格式完全一致, 只是源 MAC 地址和目的 MAC 地址换了个顺序, 如图12所示。

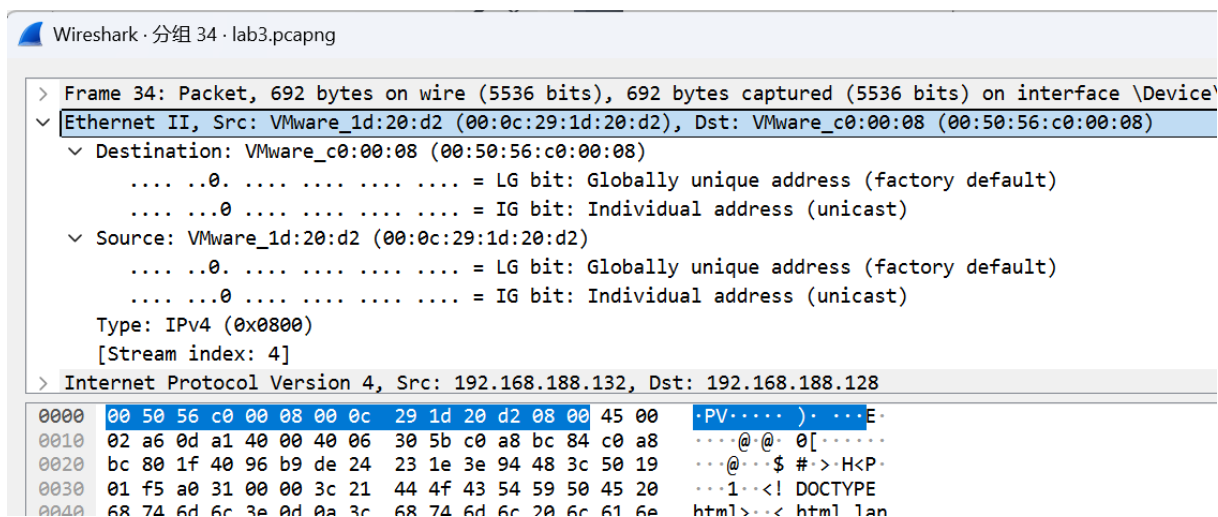


图 12: HTTP 响应包的 Ethernet II 帧头

3.2 互联层

查看 HTTP 请求包的网络层封装, 标蓝的 20 字节属于 IP 数据包的固定首部部分, 如图13所示。

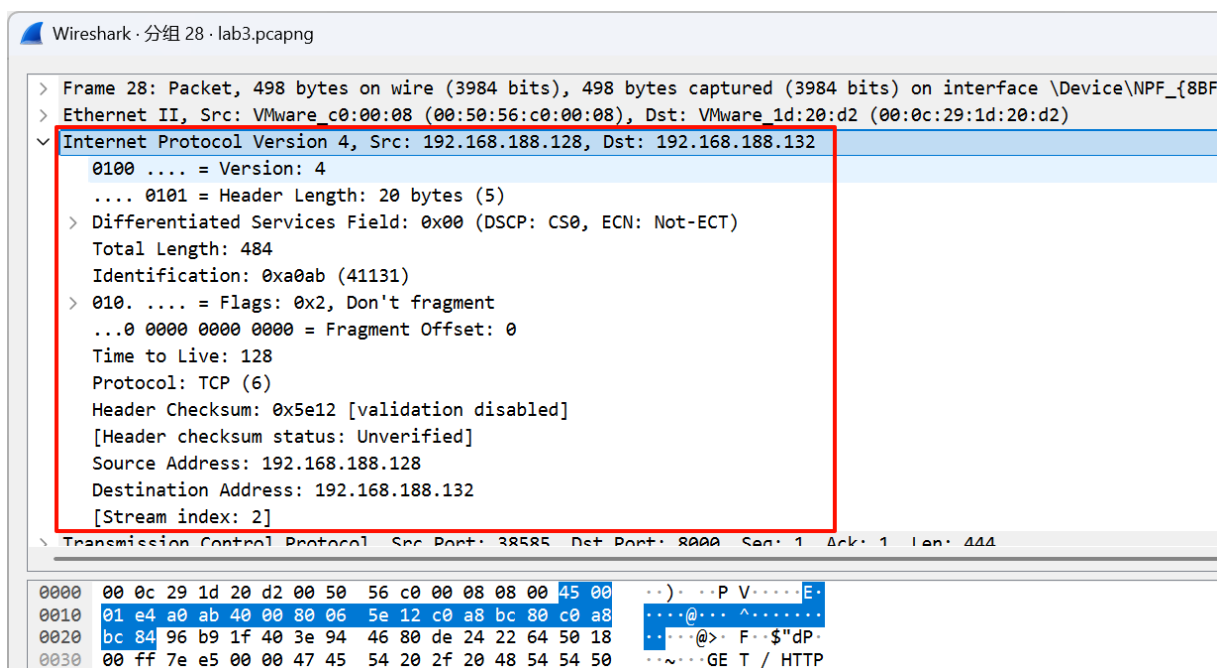


图 13: HTTP 请求包的 IP 固定首部

Version: 4表示 IP 协议版本号为 IPv4。

Header Length: 20 bytes (5)表示 IP 首部长度的, 这里是 20 字节。这是最小 IP 首部, 说明没有携带 IP 选项字段。

Differentiated Services Field: 0x00此处表示普通 HTTP 流量, 未使用 QoS 或拥塞标记。

Total Length: 484表示 IP 数据报的总长度, 此处 IP 首部 20 字节, 说明剩余 464 字节

为上层数据。

Identification: 0xa0ab (41131)是分片标识号, 用于在发生分片时, 标识同一个 IP 数据报。

Flags: 0x2, Don't fragment表明 DF 位 =1, 即禁止分片。

Fragment Offset: 0表示分片偏移量, 这里为 0 说明未分片。

Time to Live: 128表示 TTL (生存时间), 每经过一个路由器减 1, TTL=128 是 Windows 系统的典型初始值。

Protocol: TCP (6)指明上层协议, 6 表示 TCP, 告诉 IP 层把 payload 交给 TCP 处理。

Header Checksum: 0x5e12 [validation disabled]是 IP 首部校验和。

Source Address: 192.168.188.128代表源 IP 地址, 这里是 Windows 11 主机。

Destination Address: 192.168.188.132代表目的 IP 地址, 这里是 Ubuntu 虚拟机。

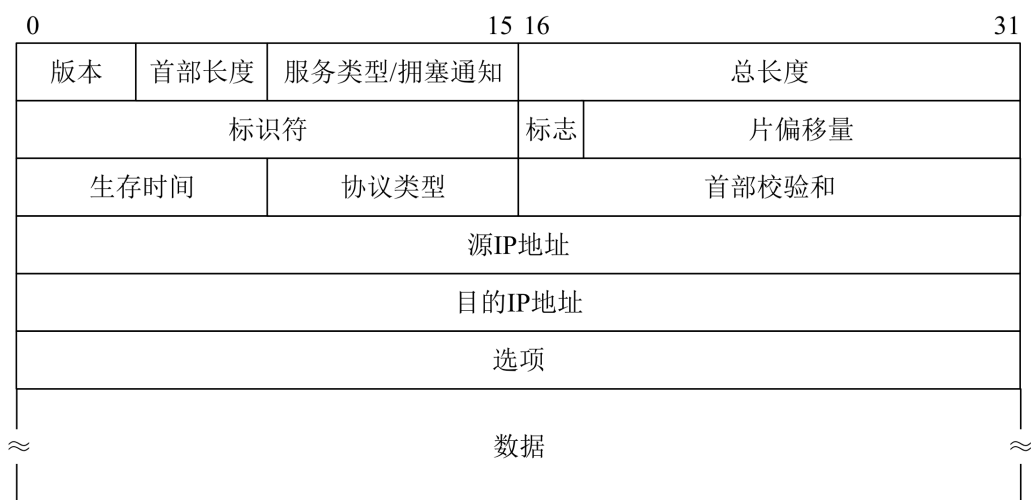


图 14: IPv4 数据包格式

查看 HTTP 响应包的网路层封装, 分析方法一模一样, 如图15所示。

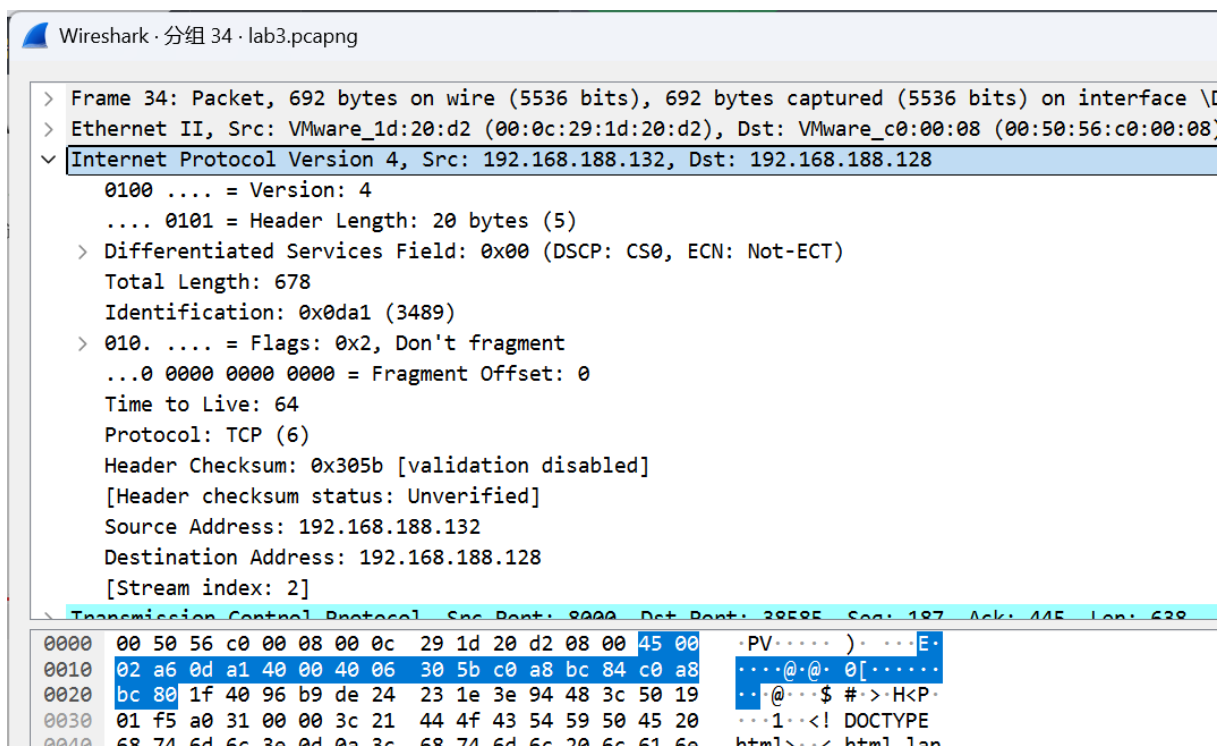


图 15: HTTP 响应包的 IP 固定首部

3.3 传输层

接着查看 HTTP 请求包的传输层封装，标蓝的 20 字节属于 TCP 首部，如图16所示。

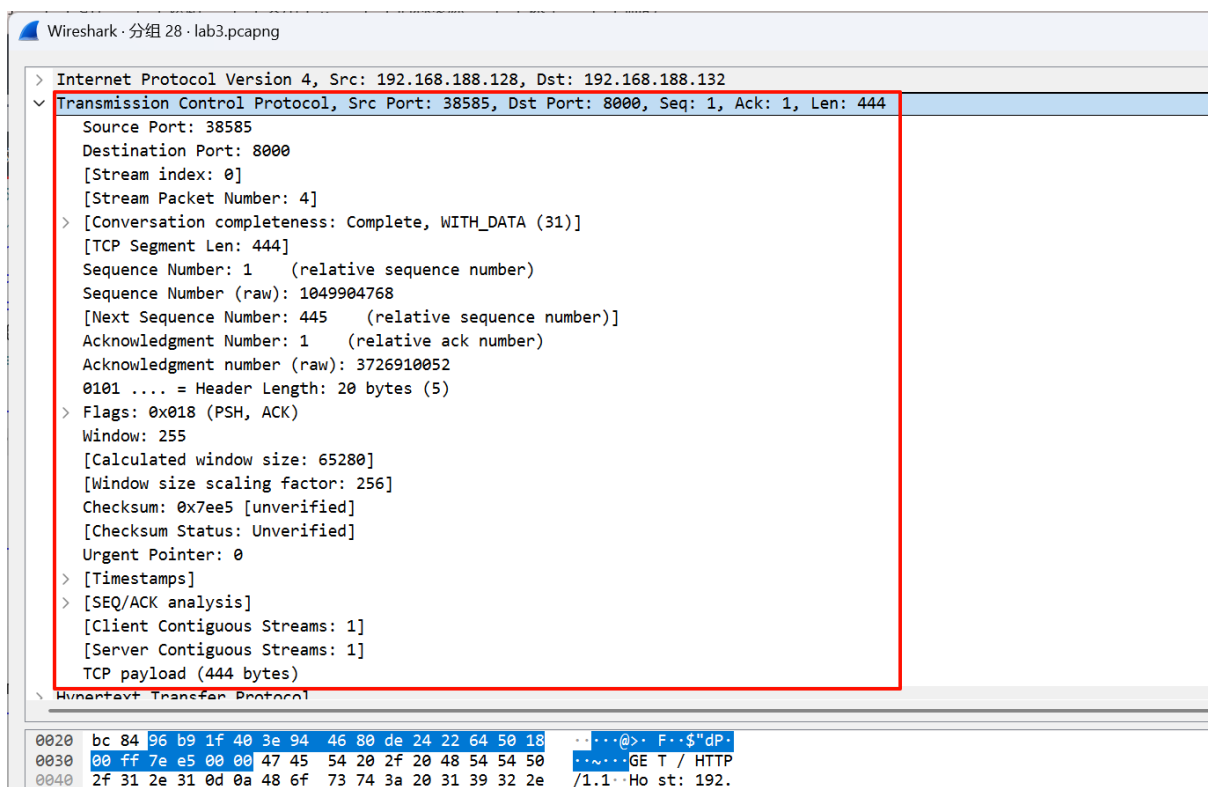


图 16: HTTP 请求包的 TCP 首部

Source Port: 38585表示源端口号，由客户端（Windows 浏览器）临时分配。

Destination Port: 8000表示目的端口号。

[TCP Segment Len: 444]指明当前 TCP 报文段中负载数据长度（不包括 TCP 首部）。

Sequence Number: 1 (relative)是 TCP 序列号，这里用的是相对序列号，真实序列号在 Sequence Number (raw) 中（这里是 1049904768）。

[Next Sequence Number: 445]表明下一个期望发送的序列号（ $1 + 444 = 445$ ，也是相对序列号）。

Acknowledgment Number: 1 (relative)是确认号，表示已经成功接收到对方序列号 < 1 的所有数据。

Header Length: 20 bytes (5)是 TCP 首部长度，20 字节表明此时未携带 TCP 选项。

Flags: 0x018 (PSH, ACK)是标志位，ACK=1 表明确认号有效，PSH=1 提示接收方尽快把数据交付给应用层。

Window: 255是 TCP 接收窗口大小。

[Calculated window size: 65280]是实际接收窗口大小（ $255 \times 256 = 65280$ ）。

[Window size scaling factor: 256]是 TCP 窗口扩大因子，在三次握手阶段协商得到。

Checksum: 0x7ee5 [unverified]是 TCP 校验和。

Urgent Pointer: 0是紧急指针，此处未使用。

TCP payload (444 bytes)是 TCP 承载的上层数据，此处就是 HTTP 请求报文。

0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
源端口号（Source Port）																目的端口号（Destination Port）															
发送序号（sequence number）																															
确认序号（length）																															
首部长度		未用				U	A	P	R	S	F	接收窗口通告（rcvr window size）																			
校验和（checksum）																紧急数据指针（ptr urgent data）															
选项（options）																															
数据（data）																															

图 17: TCP 段格式

查看 HTTP 响应包的传输层封装，分析方法一模一样，如图18所示。

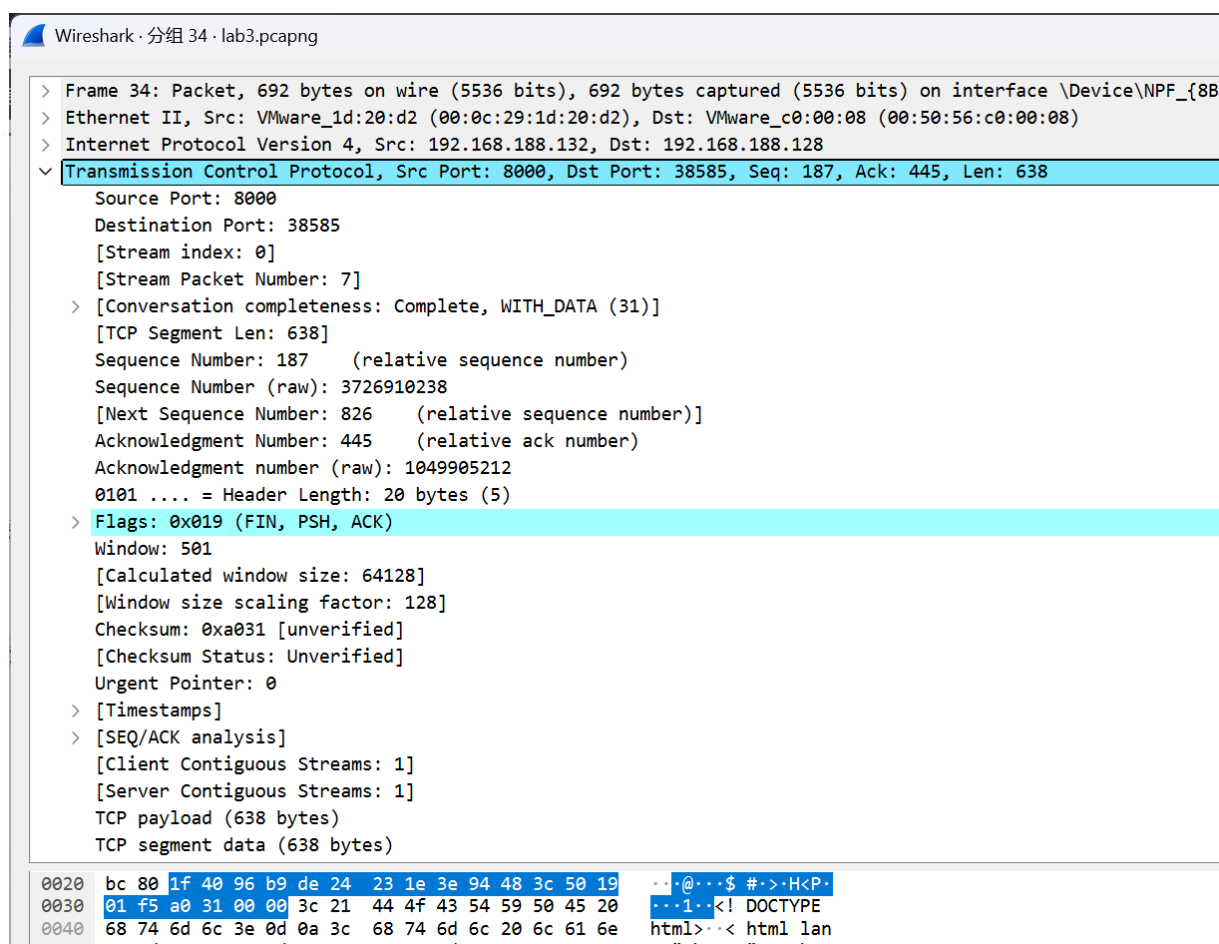


图 18: HTTP 响应包的 TCP 首部

3.4 应用层

接着查看 HTTP 请求包的应用层，也就是 HTTP 请求报文本身。HTTP 请求报文由**请求行**、**首部行**、**报文体**三部分组成，首部行和报文体之间用**空行**分隔。如图19所示。

GET / HTTP/1.1是请求行。包含请求方法（GET）、请求资源的 URL（/，也就是根目录，对应服务器上的 index.html）、HTTP 版本（HTTP/1.1）。

Host: 192.168.188.132:8000指明目标服务器的主机名和端口。

Connection: keep-alive表示请求服务器保持 TCP 连接，允许在同一 TCP 连接上发送多个 HTTP 请求。

User-Agent: Mozilla/5.0 (...) Chrome/...标识客户端软件信息，包含操作系统（Windows NT 10.0）、架构（Win64）、浏览器（Chrome）等信息。

Accept: text/html, application/xhtml+xml, ...指明浏览器可接受的响应内容类型。

Accept-Encoding: gzip, deflate指明浏览器支持的编码方案。

Accept-Language: zh-CN, zh;q=0.9, en;q=0.8指明客户端语言偏好。

\r\n是空行，非常重要，它标志 HTTP 请求首部结束。由于 GET 请求没有请求体，因此到此结束。

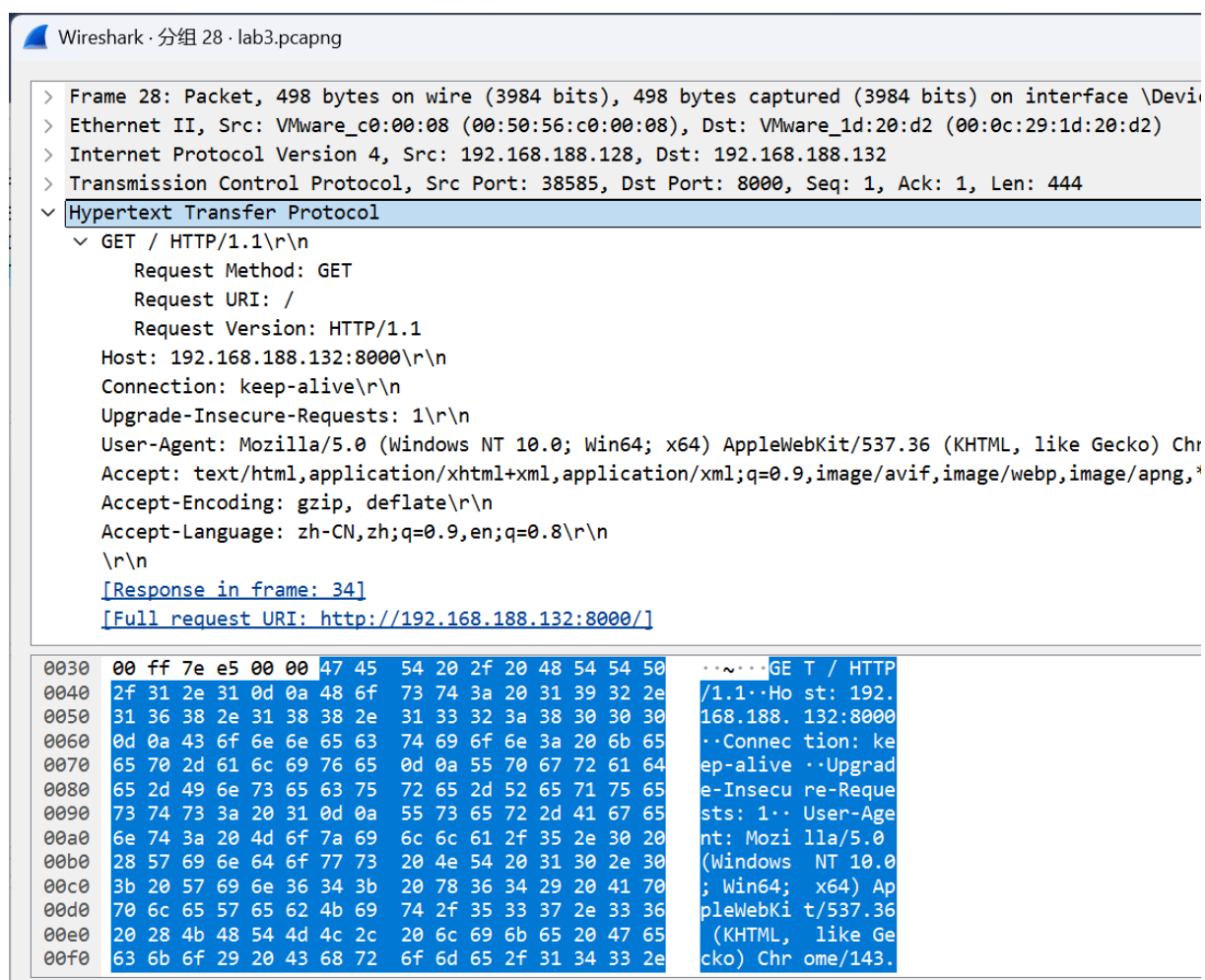


图 19: HTTP 请求报文

最后查看 HTTP 响应包的应用层，也就是 HTTP 响应报文本身。HTTP 响应报文由**状态行**、**首部行**、**报文体**三部分组成，首部行和报文体之间用空行分隔。如图20所示。

HTTP/1.0 200 OK是状态行，包含协议版本 (HTTP/1.0)、状态码 (200)、状态短语 (OK)。Server: SimpleHTTP/0.6 Python/3.12.3标识服务器软件信息 (Python 自带的 SimpleHTTPServer)。

Date: Sat, 27 Dec 2025 02:46:52 GMT是服务器发送响应的时间。

Content-Type: text/html指明响应体数据类型，告诉浏览器返回的是 HTML 文本，应当使用 HTML 解析器，而不是当成普通文件下载。

Content-Length: 638指明响应体长度 (字节，不包含响应首部)。

Last-Modified: Fri, 26 Dec 2025 17:12:24 GMT表示资源最后一次修改时间。

\r\n标志响应首部结束，后面的内容属于响应体。

File Data: 638 bytes表示 HTTP 响应体内容，即服务器返回的 HTML 页面文件内容。

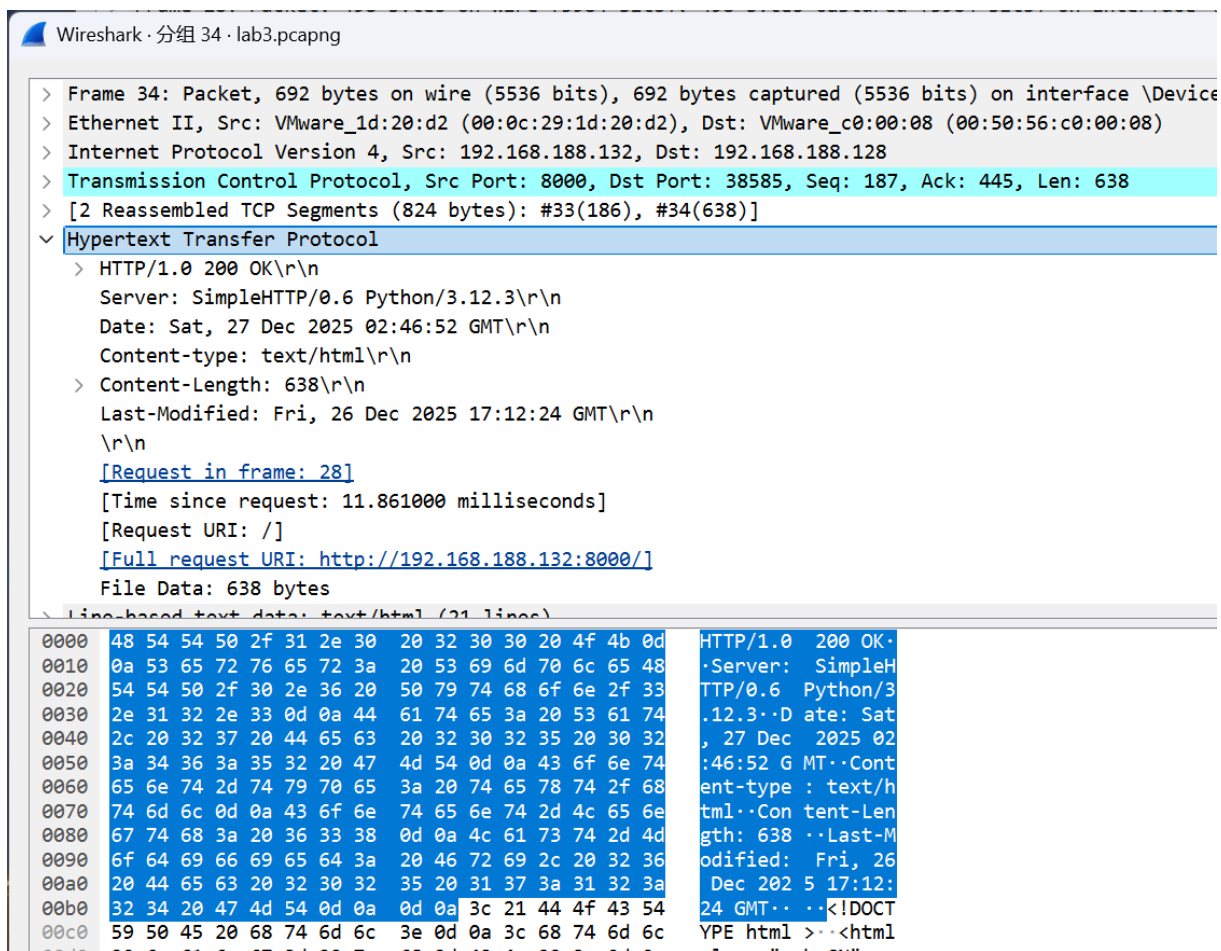


图 20: HTTP 响应报文

4 HTTP 交互过程分析

在 Wireshark 过滤栏输入http然后回车，这样只显示 HTTP 协议，如图所示。

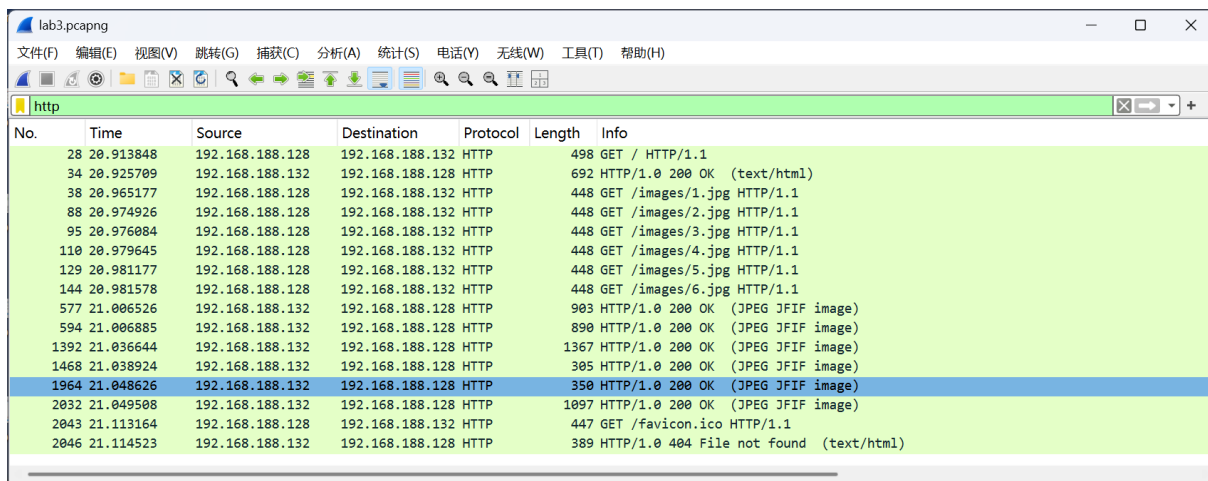


图 21: 过滤使 Wireshark 只显示 HTTP 协议

4.1 三次握手

但是在正式分析 HTTP 之前，我们应该过滤出 TCP 包进行分析，客户端首先会向 HTTP 服务器发起连接建立请求，经过三次握手之后 TCP 连接建立成功，如图22所示。

No.	Time	Source	Destination	Protocol	Length	Info
25	20.912486	192.168.188.128	192.168.188.132	TCP	66	38585 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
26	20.913506	192.168.188.132	192.168.188.128	TCP	66	8000 → 38585 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
27	20.913584	192.168.188.128	192.168.188.132	TCP	54	38585 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0

图 22: “三次握手”数据包

三次握手的具体内容已经在实验 2 编程实现了，此处不再赘述。

4.2 HTTP 交互

4.2.1 请求主页 HTML

浏览器使用GET /请求根目录,对应服务器的 index.html,并且使用Connection: keep-alive进行持久连接，如图23所示。

No.28Time20.913848Source192.168.188.128Destination192.168.188.132ProtocolHTTPLength498InfoGET / HTTP/1.1

Wireshark · 分组 28 · lab3.pcapng

> Frame 28: Packet, 498 bytes on wire (3984 bits), 498 bytes captured (3984 bits) on interface \Device\NPF{...}

> Ethernet II, Src: VMware_c0:00:08 (00:50:56:c0:00:08), Dst: VMware_1d:20:d2 (00:0c:29:1d:20:d2)

> Internet Protocol Version 4, Src: 192.168.188.128, Dst: 192.168.188.132

> Transmission Control Protocol, Src Port: 38585, Dst Port: 8000, Seq: 1, Ack: 1, Len: 444

> Hypertext Transfer Protocol

GET / HTTP/1.1\r\nRequest Method: GETRequest URI: /Request Version: HTTP/1.1Host: 192.168.188.132:8000\r\nConnection: keep-alive\r\nUpgrade-Insecure-Requests: 1\r\nUser-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.4012.91 Safari/537.36Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8Accept-Encoding: gzip, deflate\r\nAccept-Language: zh-CN,zh;q=0.9,en;q=0.8\r\n\r\n[Response in frame: 34][Full request URI: http://192.168.188.132:8000/]

图 23: 请求主页 HTML 数据包

4.2.2 服务器返回主页内容

200 OK表明服务器成功处理请求，返回 HTML 页面，内容类型为 text/html，如图24所示；浏览器解析 HTML，发现里面有标签，如图25所示。

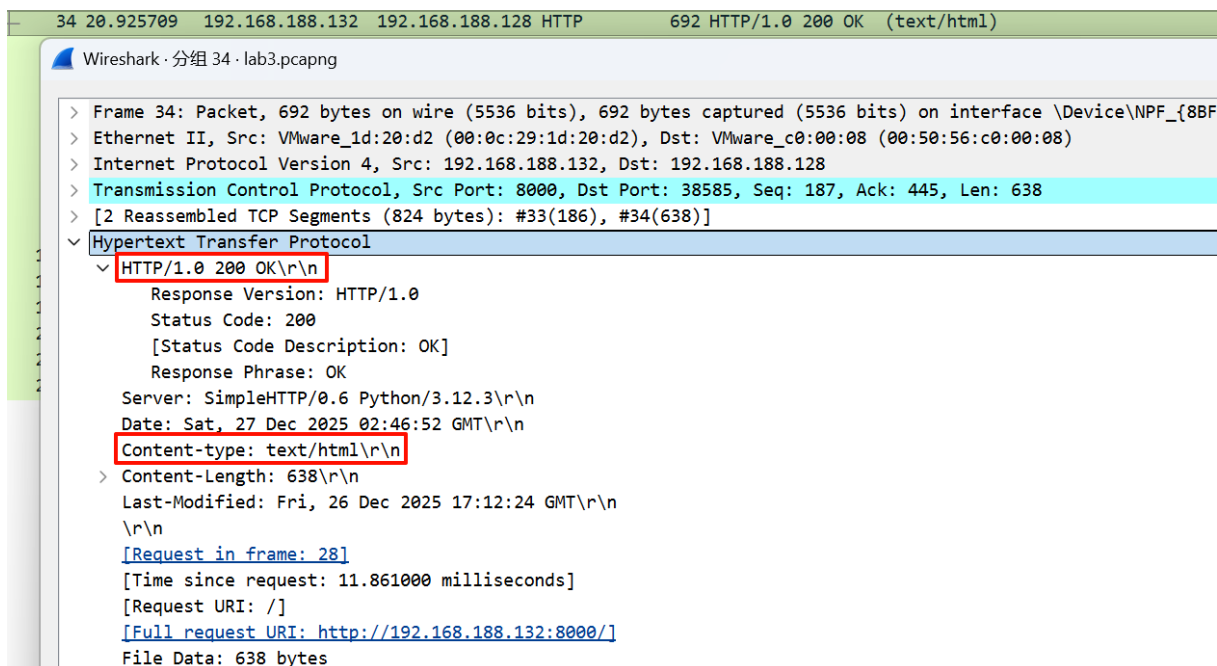


图 24: 返回主页内容数据包

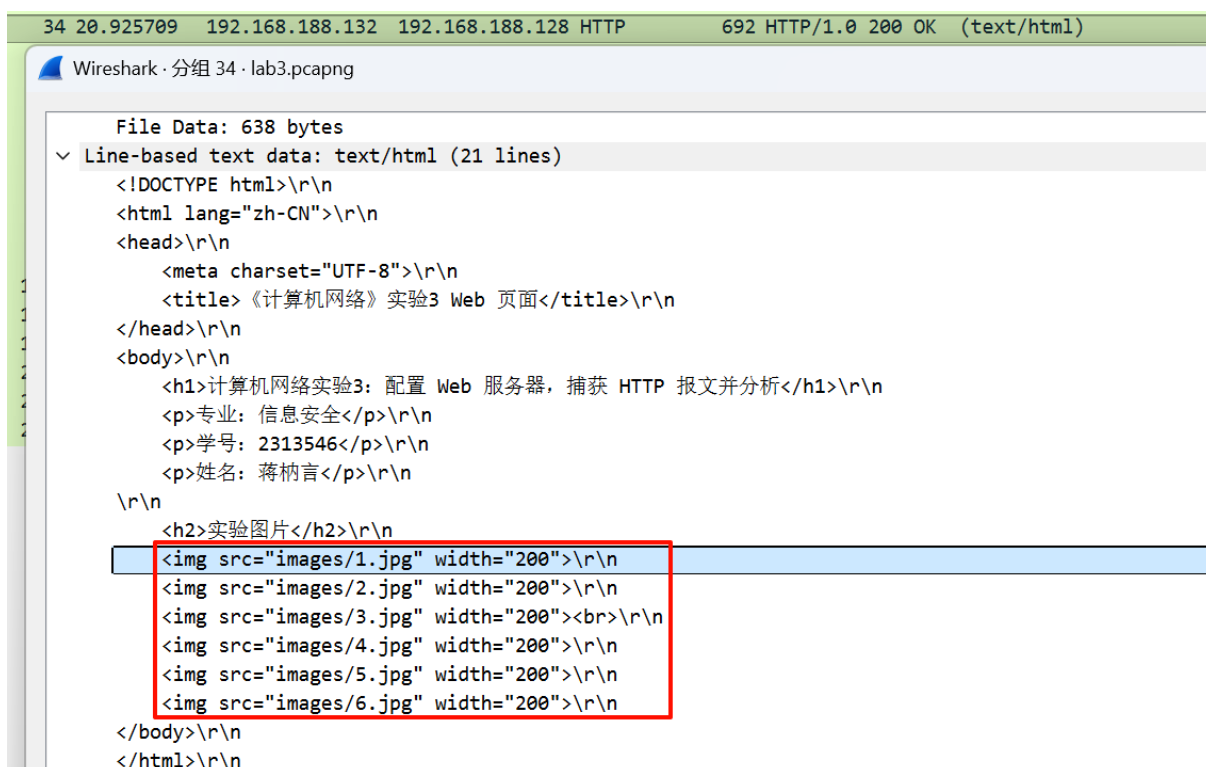


图 25: 返回主页内容数据包里的 HTML 页面包含 标签

4.2.3 浏览器为每一个图片资源发送请求

HTTP 是无状态的, 浏览器必须为每一个资源单独发请求。HTML 页面中引用了 6 张图片, 故浏览器会发送 6 个 HTTP GET 请求来分别获取 6 张图片, 如图26所示。浏览器通过多

次 HTTP GET 请求，分别获取网页所需的各个资源，最终组合渲染成完整页面。

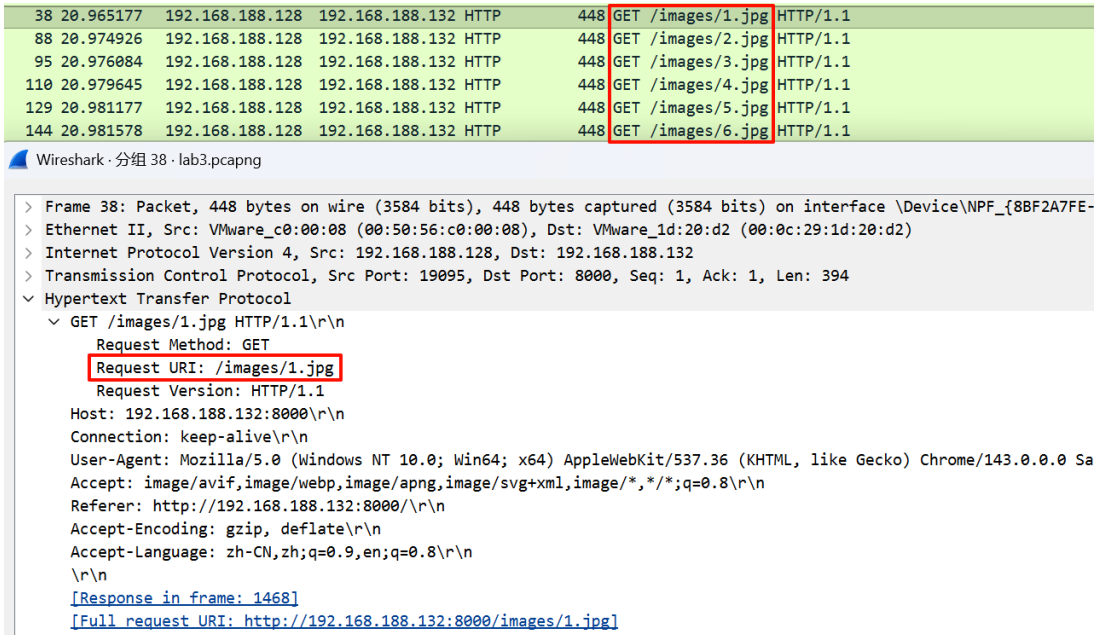


图 26: 浏览器为每一个图片资源发送请求

4.2.4 服务器逐一返回图片资源

服务器会逐一返回 6 张图片，每个请求都有独立的响应，返回内容类型为image/jpeg，如图27所示。

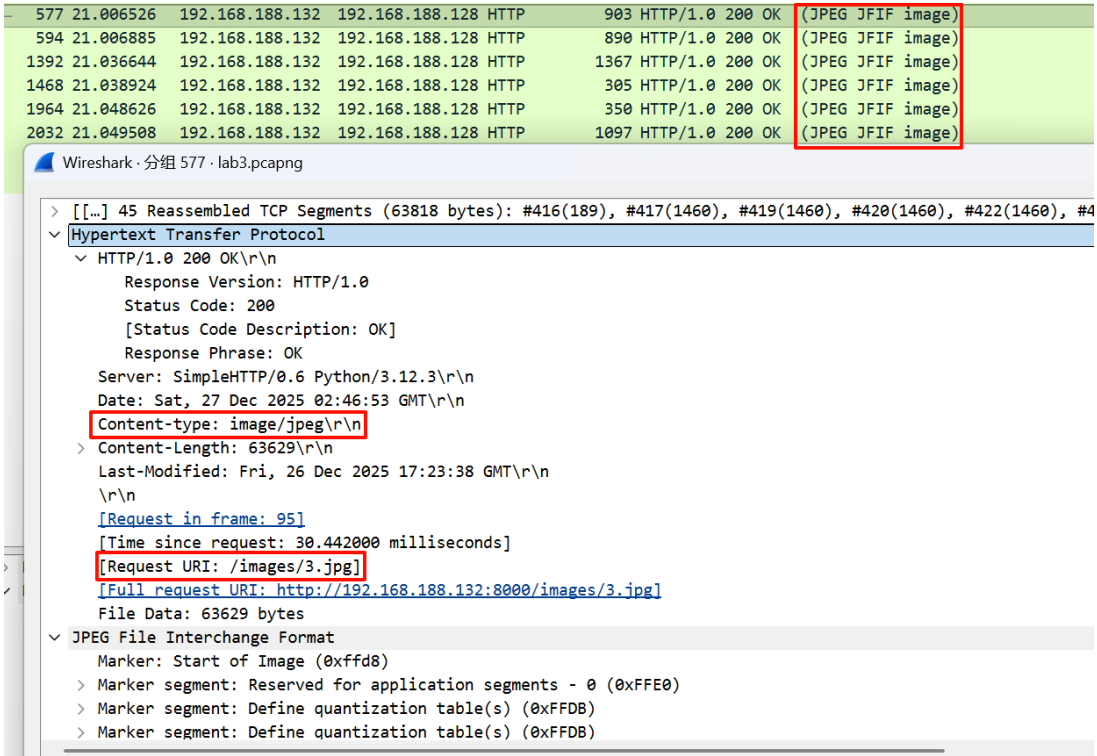


图 27: 服务器逐一返回图片资源

在这里我观察到一个**细节**，浏览器发请求的顺序是 1.jpg、2.jpg、3.jpg、4.jpg、5.jpg、6.jpg，但是服务器回复的顺序是 3.jpg、2.jpg、4.jpg、1.jpg、5.jpg、6.jpg，和请求顺序并不一致。这是因为在页面解析过程中，浏览器会并发地向服务器请求多个图片资源。由于 HTTP 请求是**并行发送**的，服务器对各请求的处理时间不同，因此返回响应的顺序可能与请求发送的顺序不一致。HTTP 协议本身**不保证响应顺序**，浏览器通过请求与响应的对应关系正确加载页面资源。

除此之外，我们还能够轻易发现不是浏览器发一个请求、服务器回一个资源，而是浏览器发请求的数据包全部在服务器响应的数据包的前面。这是使用了**流水线机制**，客户端可以将多个请求一个接一个发送出去，而不必等待前一个请求的相应报文的返回。

4.2.5 浏览器自动请求 favicon.ico，服务器返回 404 错误

最后浏览器默认会尝试获取站点图标，但是我们根本没有设置图标，服务器中不存在 favicon.ico，故返回 404 错误页面。



图 28: 浏览器请求 favicon.ico

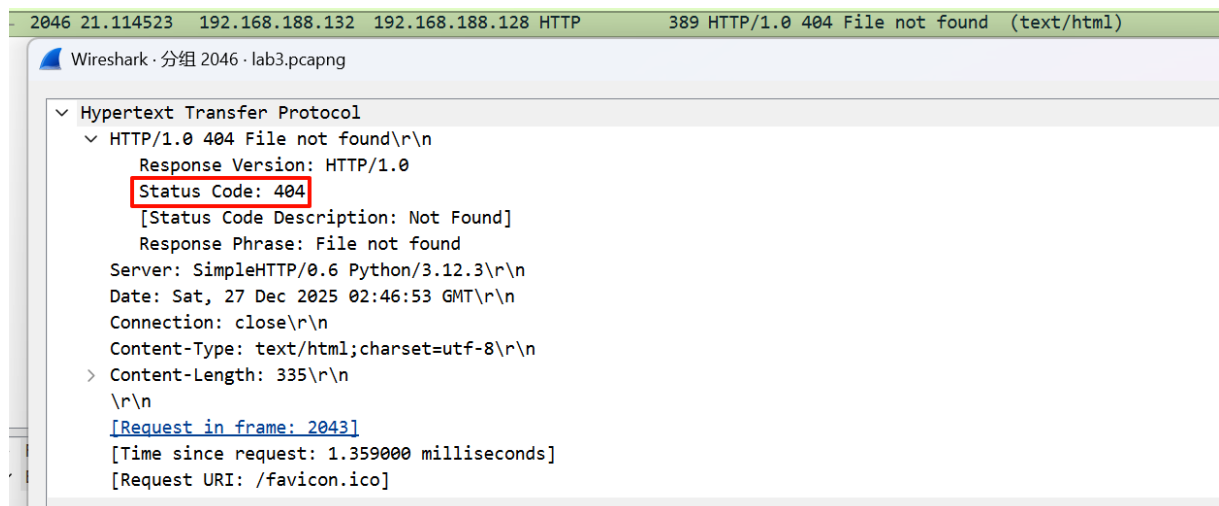


图 29: 服务器返回 404 错误

4.3 四次挥手（未观察到常规的四次挥手）

在本次实验中我未观察到常规意义的“四次挥手”过程，最后两个 TCP 数据包如图30所示。

2048	21.139859	192.168.188.128	192.168.188.132	TCP	54	10617 → 8000	[FIN, ACK]	Seq=394	Ack=522	Win=64768	Len=0
2049	21.140241	192.168.188.132	192.168.188.128	TCP	60	8000 → 10617	[ACK]	Seq=522	Ack=395	Win=64128	Len=0

图 30: 最后两个 TCP 数据包

序号为 2048 的数据包带有标志位 FIN 与 ACK，表明客户端主动关闭连接，并且确认了之前的数据；序号为 2049 的数据包带有标志位 ACK，服务器确认收到了客户端的 FIN，TCP 连接关闭。可能原因是客户端是主动关闭方，并且此时服务器已经没有数据要发，TCP 允许合并控制位，(FIN、ACK 合并成一个包)，不需要再单独一个 ACK，所以抓包中未出现典型的四个独立挥手报文。

图31对整个 HTTP 交互进行了总结。

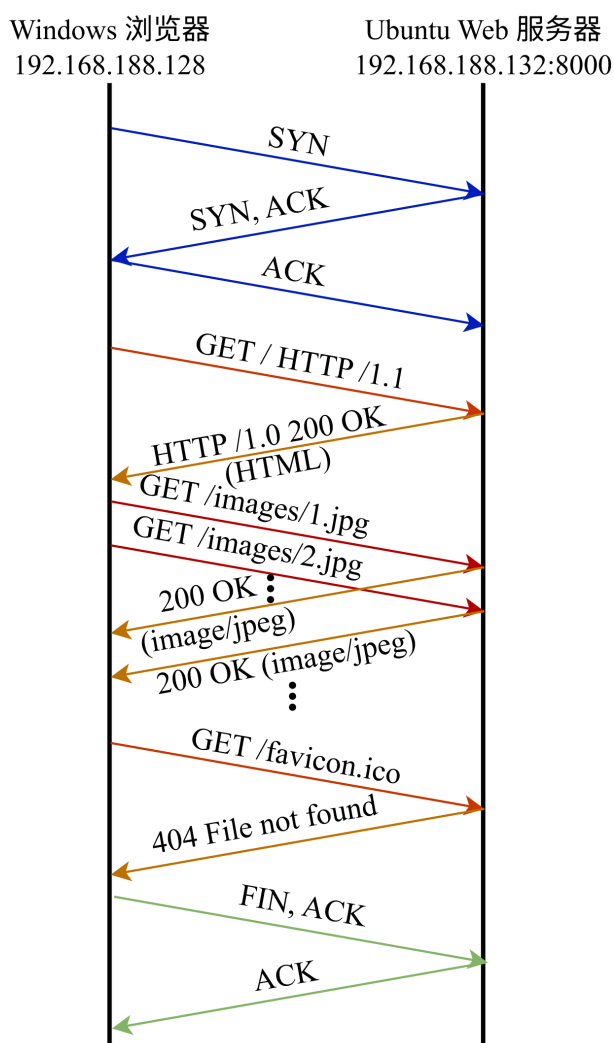


图 31: HTTP 交互示意图